

Quran Recitation AI System — Full Project Plan & Client Proposal

1. INTERNAL ENGINEERING PLAN (A → Z)

This is your complete roadmap as the AI Engineer.

A. System Overview

The system will:

- Receive **audio recitation**
 - Convert it to **text with full tashkeel**
 - Compare with the Quran text (from your DB)
 - Detect:
 - word mistakes
 - harakat mistakes
 - pronunciation mistakes
 - basic tajweed rule errors
 - Generate feedback (text + TTS)
 - Ask the user to retry
 - Evaluate improvement
-

B. Inputs You Already Have

- Full Quran text database **with tashkeel**
- 4 sheikh recordings of the full Quran
- Student audio inputs

You will use all of them in your system.

C. Architecture Overview

1. **Audio Preprocessing**
2. **Forced Alignment (Wav2Vec2)** — *Aligns audio to the expected Ayah text*
3. **Phoneme Confidence Scoring** — *Detects mispronounced letters/words*
4. **Reference Audio Alignment (DTW)** — *Aligns user audio with Sheikh audio*
5. **Prosody & Timing Comparison** — *Detects Tajweed errors via duration/pitch diffs*
6. **Error Classification**
7. **Feedback Generator (Text + TTS)**
8. **Retry System**
9. **JSON Output**
10. **API Integration**

D. Step-by-Step Engineering Plan (A → Z)

PHASE 1 — Preparation (1 Week)

Tasks:

- Normalize Quran text
- Organize Quran DB into: Surah > Ayah > Words > Harakat
- Organize 4 sheikh audio DB
- Build internal tools for:
 - audio loading
 - ayah segmentation

Deliverable: quran.json, sheikh_audio/ structured folders

PHASE 2 — Audio Pipeline (0.5 Week)

Tasks:

- Convert audio → mono, 16kHz WAV

- Apply: noise reduction, silence removal
- Build reusable audio preprocessing function

Deliverable: `preprocess_audio(audio_path)`

PHASE 3 — Forced Alignment Engine (1–2 Weeks)

Concept: Instead of trying to "guess" what the user said (ASR), we **align** their audio to the correct Ayah text. If the alignment fails or confidence is low, that's a mistake.

Tasks:

- Implement **Wav2Vec2** with a Quranic vocabulary.
- Force-align user audio to the expected text (Phoneme level).
- Extract **Confidence Scores** for every word and phoneme.

Deliverable: `align_audio(audio, expected_text) -> { word_timings, confidence_scores }`

PHASE 4 — Error Detection (Phonetic) (1 Week)

Concept: Use the confidence scores from Phase 3 to find mistakes.

Tasks:

- Thresholding: If confidence < 0.4, flag as potential error.
- Identify skipped words (duration ≈ 0).
- Identify extra words (audio segments that don't align).

Deliverable: JSON identifying "Low Confidence" words.

PHASE 5 — Reference Matching (Timing) (2–3 Weeks)

Concept: Use the Sheikh's audio as the "Gold Standard".

Tasks:

- **Dynamic Time Warping (DTW):** Align Sheikh's audio to User's audio.
- Map every phoneme in User audio to the corresponding phoneme in Sheikh audio.
- Compare durations (e.g., User held "Al-Maa" for 200ms, Sheikh held for 1000ms).

Deliverable: `compare_recordings(user_audio, sheikh_audio) -> time_diff_map`

PHASE 6 — Tajweed Analysis (Comparative) (2–3 Weeks)

Concept: Detect Tajweed errors by comparing physics (Pitch/Duration) vs Sheikh, not by hard-coding rules.

Rules to detect:

- **Madd (Elongation):** Compare duration vs Sheikh.
- **Ghunnah:** Compare nasalization (if possible) or duration.
- **Qalqalah:** Check for "plosive" energy burst at the end of letters.

Tasks:

- Build logic: `if (sheikh_duration > 0.8s) AND (user_duration < 0.3s) -> MADD_ERROR`
- Build logic: `if (sheikh_pitch_contour != user_pitch_contour) -> INTONATION_ERROR`

Deliverable: JSON with specific Tajweed advice based on the reference comparison.

PHASE 7 — Feedback System (1 Week)

Tasks:

- Explain each mistake
- Provide correct pronunciation
- Provide example audio using TTS

Example:

You said "الْحَمْدُ".
Correct is "الْحَمْدُ".
Try raising the ending with a Dammah.

PHASE 8 — Retry Logic (0.5 Week)

Tasks:

- Ask user to recite again
- Calculate improvement score

Example Output:

```
{
  "previous_score": 72,
  "current_score": 93,
  "improvement": 21
}
```

PHASE 9 — API & Documentation (1 Week)

Tasks:

- Build POST endpoint
- Input: audio file
- Output: JSON
- Write docs

PHASE 10 — Final Testing (1 Week)

Tasks:

- 40–60 audio tests
- Fix edge cases
- Final polishing

Total Internal Timeline

8–10 Weeks (Realistic for a Junior Engineer)

Internal Cost Estimate

Item	Cost
Development	\$900
Testing	\$100
GPU Compute	\$50–100

Item	Cost
Total	\$1,050 – \$1,200